

Base de datos

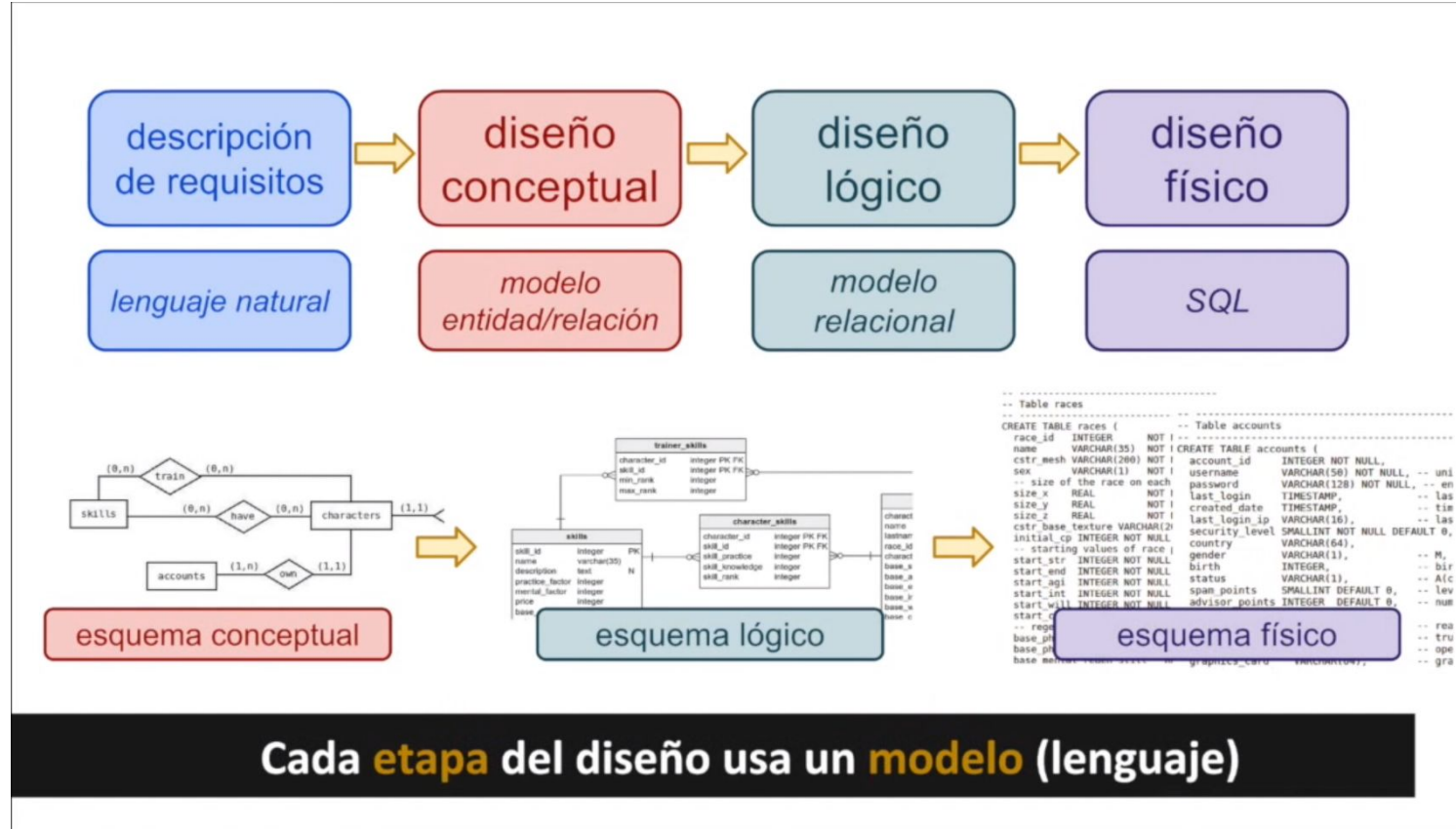


SQL

IPEAyT 186 - Capitán Luis Darío J. Castagnari (Las Higueras)

Profesor: Julián Conde

Etapas en el diseño de las BD relacionales y sus modelos



Cada **etapa** del diseño usa un **modelo** (lenguaje)

Captura del video de Marques que se encuentra más adelante.

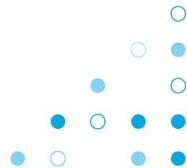
SQL (Structured Query Language, Lenguaje de consultas estructurado)

SQL: Conceptos y utilización en consultas a bases de datos.

Las tres partes de SQL:

- Lenguaje de Definición de datos (DDL),
- Lenguaje de Manipulación de datos (DML) y
- Lenguaje de Control de Datos (DCL).

MySQL: Gestor de Bases de Datos Relacionales



Control de Instalación de SGBD MySQL y App gráficas

SGBD que nosotros vemos es MySQL. Controlemos que tenemos instalado.

XAMPP Server

Ver si está la carpeta

D:/xampp



The screenshot shows the XAMPP website homepage. At the top is a dark blue navigation bar with links: Apache Friends, Descargar, Alojamiento, Comunidad, and Acerca de. There is a search bar with the text 'Buscar..' and a green 'Buscar' button. A language selector shows 'ES' with a dropdown arrow. Below the navigation bar, the main heading reads 'XAMPP Apache + MariaDB + PHP + Perl' with the XAMPP logo. Underneath, a section titled '¿Qué es XAMPP?' explains that XAMPP is the most popular environment for PHP development, a free and easy-to-install distribution of Apache containing MariaDB, PHP, and Perl. To the right of this text is a large image of the XAMPP logo, which features a play button icon inside an orange square. At the bottom, there are four download buttons: a large green 'Descargar' button with the text 'Pulsa aquí para otras versiones', and three smaller buttons for 'XAMPP para Windows 8.2.12 (PHP 8.2.12)', 'XAMPP para Linux 8.2.12 (PHP 8.2.12)', and 'XAMPP para OS X 8.2.4 (PHP 8.2.4)'.

Instalamos XAMPP Server

En esta dirección

<https://www.apachefriends.org/es/index.html>

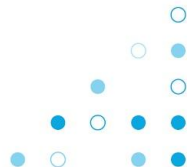
podemos descargar xampp server, es un instalador de varias tecnologías, entre ellas: PHP, Servidor Web Apache, MySQL y **PhpMyAdmin**.

XAMPP es una compilación de software libre.

XAMPP está en las compu del laboratorio y en algunas notebook.



¿SE PUEDE INSTALAR CADA SOFTWARE POR SEPARADO?
SI.
¿COMO?



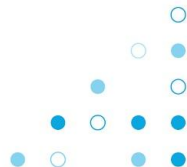
Uso de XAMPP Server

Si lo tenemos instalado, cada vez que usemos el servidor utilizando phpMyAdmin o Workbench, o una aplicación web que está hecha en php, debemos:

1. Ir hasta la carpeta de **xampp** (d:/xampp)
2. Ejecutar **xampp_control.exe** e iniciar apache y mysql
3. Trabajar con mi proyecto de php que se encuentre ubicado en **d:/xampp/htdocs/mi_proyecto**
4. Abrir **navegador** y poner **localhost/mi_proyecto**
5. Para phpMyAdmin deben abrir un **navegador** y poner **localhost/phpmyadmin**



The screenshot shows the XAMPP website interface. At the top, there's a navigation bar with links: Apache Friends, Descargar, Alojamiento, Comunidad, and Acerca de. A search bar is on the right. The main heading is "XAMPP Apache + MariaDB + PHP + Perl". Below this, a section titled "¿Qué es XAMPP?" explains that it's the most popular development environment with PHP, distributed by Apache, and is free and easy to install. To the right is the XAMPP logo. Below the text are three download buttons: "Descargar" (with a subtext "Pulsa aquí para otras versiones"), "XAMPP para Windows 8.2.12 (PHP 8.2.12)", "XAMPP para Linux 8.2.12 (PHP 8.2.12)", and "XAMPP para OS X 8.2.4 (PHP 8.2.4)". At the bottom, a "New XAMPP release 8.2.12, 8.1.25 and 8.0.30" announcement states that new versions for Windows, Linux, and OS X are available. A "Más información »" link is provided.



Instalación y uso de SGBD – Clientes gráficos de SGBD

SGBD que nosotros vemos es MySQL. Controlemos que tenemos instalado.

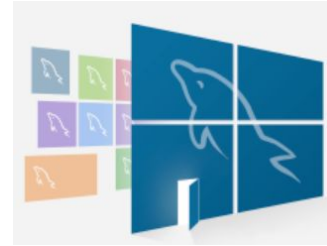
Instalemos desde MySQL Community Downloads: <https://dev.mysql.com/downloads/>

MySQL Server (es el SGBD, el motor de BD)

y luego para trabajar más cómodos:

Cliente gráfico de MySQL:

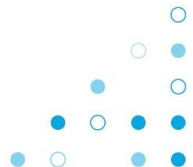
Aplicación de escritorio: **MySQL Workbench**
(requiere mysql server)



Aplicación web: **phpMyAdmin**
(requiere, mysql server php y servidor web apache)



VEAMOS CADA UNO DE ESTAS DOS APLICACIONES



Instalación y uso de SGBD – Clientes gráficos de SGBD

Dirección web: <https://dev.mysql.com/downloads/>

MySQL Community Downloads

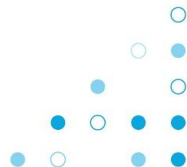
- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL NDB Cluster
- MySQL Router
- MySQL Shell
- MySQL Operator
- MySQL NDB Operator
- MySQL Workbench
- MySQL Installer for Windows
- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

MySQL Enterprise Edition for Developers

Free for learning, developing,
and prototyping.

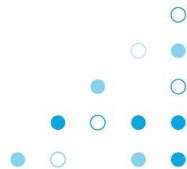


Download Now »



Instalación y uso de SGBD – Clientes gráficos de SGBD

1. Veamos **XAMPP Server**
2. Ver que hay en la carpeta **xampp**
3. Iniciar **xampp_control.exe** y analizar lo que tiene ese panel
4. Ingresar y analizar **phpMyAdmin** (app web gráfica de SGBD)
5. ¿Donde debería tener mi proyecto web con **php**?
6. Ingresar y analizar **MySQL Workbench** (app escritorio gráfica de SGBD)





SQL

SQL

SQL es una herramienta para organizar, gestionar y recuperar datos almacenados en una base de datos relacional, "SQL" es una abreviatura de Structured Query Language (Lenguaje de Consultas Estructurado).

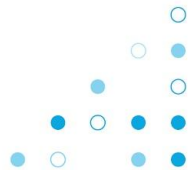


Historia del Estándar SQL

El **Lenguaje SQL (Structured Query Language)** tiene su origen en la década de 1970, dentro de los laboratorios de **IBM**, donde se desarrolló un sistema experimental de gestión de bases de datos llamado **System R**.

En este proyecto surgió un lenguaje de consulta denominado **SEQUEL** (Structured English Query Language), que más tarde dio lugar al actual **SQL**.

En **1986**, el **ANSI (American National Standards Institute)** adoptó SQL como el **lenguaje estándar** para la manipulación de bases de datos relacionales. Poco después, la **ISO (International Organization for Standardization)** también lo reconoció oficialmente.



Historia del Estándar SQL

A lo largo del tiempo se fueron incorporando nuevas versiones del estándar:

- **SQL-89** y **SQL-92** agregaron más tipos de datos y control de integridad.
- **SQL:1999** introdujo **procedimientos almacenados**, **triggers** y **tipos de datos definidos por el usuario**.
- **SQL:2003** amplió el soporte para **XML**.
- **SQL:2011** incorporó manejo de **datos temporales**.
- **SQL:2016** añadió soporte para **JSON**, muy usado actualmente.

Aunque **ANSI/ISO SQL** define el estándar, **cada empresa** adaptó su propia versión:

- **Oracle** desarrolló **PL/SQL**.
- **Microsoft** lanzó **Transact-SQL (T-SQL)**.
- **PostgreSQL** mantiene gran compatibilidad con el estándar y es de **código abierto**.
- **MySQL**, también de **libre distribución**, se convirtió en uno de los **servidores más usados en Internet**, especialmente en entornos **web**, junto con **MariaDB** (su fork comunitario).

Hoy SQL es un **lenguaje universal** para trabajar con bases de datos relacionales, tanto locales como en la nube.



Lenguaje de consulta SQL

Structured Query Language
Lenguaje de Consulta
Estructurado

SQL brinda diferentes sentencias para definir datos, realizar consultas y actualizaciones.

Tutorial útil:

www.w3schools.com/sql/default.asp

Lenguaje de definición de datos: DDL (Data Definition Language). **Permiten crear y definir nuevas bases de datos, campos e índices**

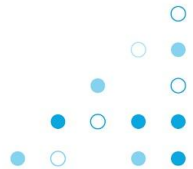
Lenguaje de manipulación de datos: DML (Data Manipulation Language). **Permiten generar consultas para ordenar, filtrar y extraer datos de la base de datos, además incluye de sentencias para agregar modificar y eliminar datos**

Lenguaje de Control de Datos: DCL (Data Control Language): **permite manipular los permisos sobre las bases de datos y sus objetos**

Lenguaje de consulta SQL

DDL

- ✓ **CREATE:** Crea nuevas tablas, campos e índices.
- ✓ **DROP:** Elimina tablas e índices.
- ✓ **ALTER:** Modifica las tablas agregando campos o cambiando la definición de los campos.



Lenguaje de consulta SQL

SQL utiliza los términos tabla, fila y columna para los términos relación, tupla y atributo del modelo relacional formal, respectivamente.

SQL es un lenguaje de bases de datos global: cuenta con sentencias para definir datos, consultas y actualizaciones.

A tener en cuenta en su sintaxis que se utiliza para mostrar sus distintas sentencias:

{**Alternativas**}, entre llaves se colocarán las palabras que tienen opciones o alternativas en la sentencia a la que pertenecen.

[**Opcional**], entre corchetes se colocarán las palabras que son opcionales en la sentencia, es decir que pueden colocarse o pueden obviarse.



Lenguaje de consulta SQL

DDL

Create Database

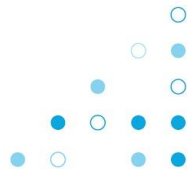
```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] nombre_base_datos
```

Create Table

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] nombre_de_tabla  
nombre_de_columna tipo_de_dato [NOT NULL | NULL] [DEFAULT valor_por_defecto]  
[AUTO_INCREMENT] [UNIQUE | [PRIMARY] KEY  
[CONSTRAINT [nombre_relación] FOREIGN KEY (nombre_columna)  
REFERENCES nombre_de_tabla (nombre_columna)]  
[ON DELETE opciones_de_referencia]  
[ON UPDATE opciones_de_referencia]
```

Create Index

```
CREATE INDEX nombre_indice  
ON nombre_tabla (col_name [(length)] [ASC | DESC])  
[opcion_algoritmo | opcion_bloqueo] ...
```



Lenguaje de consulta SQL

DDL

Create Table

Ejemplo

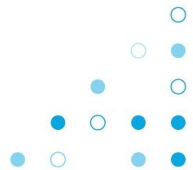
```
CREATE TABLE `Products` (  
  `ProductID` int NOT NULL,  
  `ProductName` varchar(45) DEFAULT NULL,  
  `SupplierID` int DEFAULT NULL,  
  `CategoryID` int DEFAULT NULL,  
  `Unit` varchar(255) DEFAULT NULL,  
  `Price` double DEFAULT NULL,  
  PRIMARY KEY (`ProductID`),  
  KEY `fk_Products_1_idx` (`CategoryID`),  
  KEY `fk_Products_2_idx` (`SupplierID`),  
  CONSTRAINT `fk_Products_1` FOREIGN KEY (`CategoryID`) REFERENCES `Categories` (`CategoryID`),  
  CONSTRAINT `fk_Products_2` FOREIGN KEY (`SupplierID`) REFERENCES `Suppliers` (`SupplierID`)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```



Ejemplo

```
CREATE TABLE Proveedores  
(Nprov INTEGER NOT NULL AUTO_INCREMENT  
CONSTRAINT pkproveedores PRIMARY KEY,  
Nombre VARCHAR(50) NOT NULL CONSTRAINT uniconombre  
UNIQUE,  
Direccion VARCHAR(50));
```

La ejecución de este comando produce como resultado la creación de una tabla Proveedores con tres campos; Nprov: de tipo entero y clave primaria de la tabla y autoincrementado, Nombre: de tipo cadena de caracteres de 50 e índice único o clave secundaria, y por último el campo Dirección de tipo cadena de caracteres de 50.

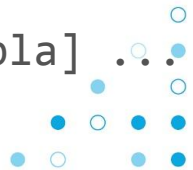


Alter Table

```
ALTER [IGNORE] TABLE nombre_tabla
ADD [COLUMN] nombre_columna columna_definicion
[FIRST | AFTER col_nombre]
| ADD [COLUMN] (nombre_columna definición_columna)
| ADD {INDEX|KEY} [nombre_indice]
[tipo_indice] (nombre_columna_indice) [indice_opcion] ...
| ADD [CONSTRAINT] PRIMARY KEY
[indice_type] (indice_col_nombre,...) [indice_opcion] ...
| ADD [CONSTRAINT [nombre_relación] FOREIGN KEY (nombre_columna)
REFERENCES nombre_de_tabla (nombre_columna)]
```

Drop Table

```
DROP [TEMPORARY] TABLE [IF EXISTS] nombre_tabla [,nombre_tabla] ...
```



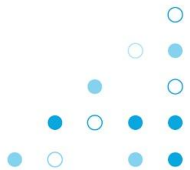
Lenguaje de consulta SQL

DDL

Alter Table

Ejemplo

```
ALTER TABLE `w3schools`.`Products`  
ADD COLUMN `observacion` VARCHAR(45) NULL AFTER `Price`;
```



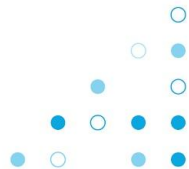
Tipo de Datos

Los tipos de datos SQL se clasifican en 13 categorías primarias, las cuales pueden tener sinónimos reconocidos por dichos tipos de datos.

Dependiendo del manejador de base de datos, puede haber algunas variaciones en los tipos de datos de los atributos o campos.

En SQL-92, además de los tipos de datos del SQL-89 (INTEGER, SMALLINT, CHARACTER, DECIMAL, NUMERIC, REAL, FLOAT y DOUBLE PRECISION) también se admiten los siguientes: CHARACTER VARYING, DATE, TIME, BIT, TIMESTAMP, INTERVAL y BIT.

SQL 99, extiende SQL con características de base de datos objeto-relacionales.



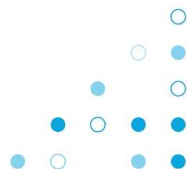
Tipo de Datos

1. Números

Enteros

Tipo	Rango (con signo)	Ejemplo
TINYINT	-128 a 127	<code>edad = 25</code>
SMALLINT	-32,768 a 32,767	<code>año_fabricacion = 1999</code>
MEDIUMINT	-8M a 8M aprox.	<code>poblacion = 7500000</code>
INT / INTEGER	-2,147,483,648 a 2,147,483,647	<code>dni = 12345678</code>
BIGINT	-9 cuatrillones a 9 cuatrillones	<code>telefono = 5493515551234</code>

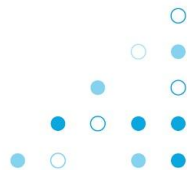
 Se pueden declarar como **UNSIGNED** (solo positivos, duplica rango positivo).



Tipo de Datos

Decimales y punto flotante

Tipo	Precisión	Ejemplo
DECIMAL(p,s)	Número exacto con p dígitos totales y s decimales	precio DECIMAL(8,2) = 12345.67
NUMERIC(p,s)	Igual a DECIMAL	saldo = 1500.50
FLOAT	Aproximado (precisión simple)	temperatura = 36.6
DOUBLE / REAL	Aproximado (doble precisión)	altura = 1.75234



Tipo de Datos

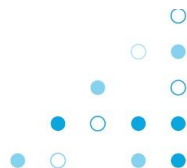
2. Textos y cadenas

Cadenas de longitud fija

Tipo	Descripción	Ejemplo
CHAR(n)	Siempre ocupa n caracteres (rellena con espacios si es corto)	<code>codigo CHAR(5) = 'A12 '</code>

Cadenas de longitud variable

Tipo	Descripción	Ejemplo
VARCHAR(n)	Hasta n caracteres (solo ocupa lo que se use)	<code>nombre VARCHAR(50) = 'María'</code>



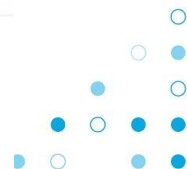
Tipo de Datos

Textos largos

Tipo	Tamaño máximo	Ejemplo
TINYTEXT	255 caracteres	<code>comentario = 'OK'</code>
TEXT	65,535 caracteres (~64 KB)	<code>descripcion = 'Lorem ipsum ...'</code>
MEDIUMTEXT	16 millones de caracteres	<code>articulo = 'Texto muy largo...'</code>
LONGTEXT	4 mil millones de caracteres	<code>documento = '...'</code>

Binarios (datos en crudo)

Tipo	Descripción	Ejemplo
BLOB (Binary Large Object)	Almacena binarios (archivos, imágenes)	<code>imagen BLOB</code>
TINYBLOB / MEDIUMBLOB / LONGBLOB	Variantes según tamaño	archivo PDF o JPG



Tipo de Datos

3. Fechas y horas

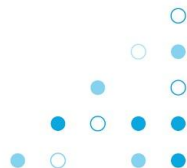
Tipo	Formato	Ejemplo
DATE	YYYY-MM-DD	<code>fecha_nac = '1990-07-25'</code>
DATETIME	YYYY-MM-DD HH:MM:SS	<code>created_at = '2025-09-08 12:30:00'</code>
TIMESTAMP	Igual a DATETIME, pero cambia con zona horaria y se actualiza automáticamente	<code>last_login = '2025-09-08 12:30:00'</code>
TIME	HH:MM:SS	<code>hora = '14:45:10'</code>
YEAR	YYYY	<code>año = 2025</code>



Tipo de Datos

4. Booleanos y enumeraciones

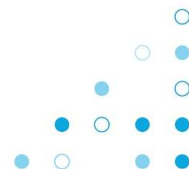
Tipo	Descripción	Ejemplo
BOOLEAN / BOOL	Alias de <code>TINYINT(1)</code> (0 = false, 1 = true)	<code>activo = TRUE</code>
ENUM	Lista de valores predefinidos (uno solo)	<code>sexo ENUM('M', 'F', 'X') = 'F'</code>
SET	Lista de valores predefinidos (pueden ser varios)	<code>dias_laborales</code> <code>SET('Lunes', 'Martes', 'Miércoles') = 'Lunes, Miércoles'</code>



Tipo de Datos

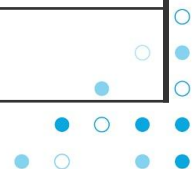
 En resumen:

- **Números** → INT , BIGINT , DECIMAL , FLOAT , DOUBLE
- **Textos** → CHAR , VARCHAR , TEXT , BLOB
- **Fechas** → DATE , DATETIME , TIMESTAMP , TIME , YEAR
- **Booleanos y enumeraciones** → BOOLEAN , ENUM , SET



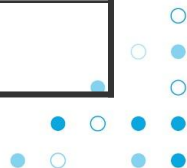
Tipos de datos de MySQL

TinyInt	Es un número entero con o sin signo. Con signo el rango de valores válidos va desde -128 a 127. Sin signo, el rango de valores es de 0 a 255.
Bit ó Bool	Un número entero que puede ser 0 ó 1.
SmallInt	Número entero con o sin signo. Con signo el rango de valores va desde -32768 a 32767. Sin signo, el rango de valores es de 0 a 65535.
MediumInt	Número entero con o sin signo. Con signo el rango de valores va desde -8.388.608 a 8.388.607. Sin signo el rango va desde 0 a 16777215.
Integer, Int:	Número entero con o sin signo. Con signo el rango de valores va desde -2147483648 a 2147483647. Sin signo el rango va desde 0 a 429.4967.295.
BigInt	Número entero con o sin signo. Con signo el rango de valores va desde -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807. Sin signo el rango va desde 0 a 18.446.744.073.709.551.615.
Float	Número pequeño en coma flotante de precisión simple. Los valores válidos van desde -3.402823466E+38 a -1.175494351E-38, 0 y desde 1.175494351E-38 a 3.402823466E+38 .
xReal, Double	Número en coma flotante de precisión doble. Los valores permitidos van desde -1.7976931348623157E+308 a -2.2250738585072014E-308, 0 y desde 2.2250738585072014E-308 a 1.7976931348623157E+308 .
Decimal, Dec, Numeric	Número en coma flotante desempaquetado. El número se almacena como una cadena.



Tipos de datos de MySQL

Date	Tipo fecha, almacena una fecha. El rango de valores va desde el 1 de enero del 1001 al 31 de diciembre de 9999. El formato de almacenamiento es de año-mes-día.
DateTime	Combinación de fecha y hora. El rango de valores va desde el 1 de enero del 1001 a las 0 horas, 0 minutos y 0 segundos al 31 de diciembre del 9999 a las 23 horas, 59 minutos y 59 segundos. El formato de almacenamiento es de año-mes-día horas:minutos:segundos.
TimeStamp	Combinación de fecha y hora. El rango va desde el 1 de enero de 1970 al año 2037. El formato de almacenamiento depende del tamaño del campo.
Char(n)	Almacena una cadena de longitud fija. La cadena podrá contener desde 0 a 255 caracteres.
VarChar(n)	Almacena una cadena de longitud variable. La cadena podrá contener desde 0 a 255 caracteres.
TinyText y TinyBlob	Columna con una longitud máxima de 255 bytes (caracteres en text)..
Blob y Text	Columna con un máximo de 65535 bytes (caracteres en text)..
MediumBlob y MediumText	Columna con un máximo de 16.777.215 bytes (caracteres en text).
LongBlob y LongText	Columna con un máximo de caracteres 4.294.967.295 bytes (caracteres en text).

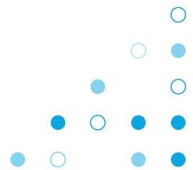


Ligaduras de Integridad

- Claves primarias (Primary key)
- Claves secundarias o candidatas (Unique)
- **Integridad referencial (Foreign key)**

Se utiliza la cláusula **CONSTRAINT** en las instrucciones ALTER TABLE y CREATE TABLE para crear o eliminar restricciones.

Existen dos sintaxis para esta cláusula dependiendo si se quiere crear un índice de un único campo o si se trata de un índice multi-campo.



Cláusula Constraint

Para los índices de campos únicos:

CONSTRAINT *nombre*

{PRIMARY KEY | UNIQUE | NOT NULL | FOREIGN KEY (*referencia1*) REFERENCES *tablaexterna* [(*campoexterno1*)]}

Para los índices de múltiples campos:

CONSTRAINT *nombre*

{

PRIMARY KEY (*primario1* [, *primario2* [, ...]])

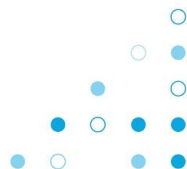
| UNIQUE (*único1* [, *único2* [, ...]])

| NOT NULL (*nonulo1* [, *nonulo2* [, ...]])

| FOREIGN KEY (*referencia1* [, *referencia2* [, ...]]) REFERENCES *tablaexterna* [(*campoexterno1* [, *campoexterno2* [, ...]])]

[ON DELETE *accion*][ON UPDATE *accion*]

}



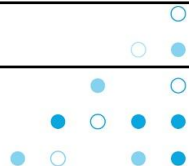
Argumentos de la cláusula CONSTRAINT

Tipo de Índice	Descripción
PRIMARY KEY	Genera un índice primario el campo o los campos especificados. Todos los campos de la clave primaria deben ser únicos y no nulos, cada tabla sólo puede contener una única clave primaria.
UNIQUE	Genera un índice de clave única, de esta forma se generan las claves secundarias o candidatas.
CHECK	Es para especificar una condición (condición) que deben cumplir los datos de una tabla.
FOREIGN KEY	Genera un índice externo (toma como valor del índice campos contenidos en otras tablas), de esta forma se incorporan restricciones de integridad referencial .



Argumentos de la cláusula CONSTRAINT

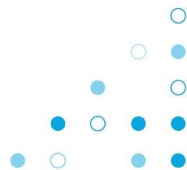
Argumento	Descripción
<i>Nombre</i>	Es el nombre del índice que se va a crear.
<i>PrimarioN</i>	Es el nombre del campo o de los campos que forman el índice primario.
<i>UnicoN</i>	Es el nombre del campo o de los campos que forman el índice de clave única.
<i>NonuloN</i>	Es el nombre del campo o de los campos que no admiten valores nulos.
<i>ReferenciaN</i>	Es el nombre del campo o de los campos que forman el índice externo (hacen referencia a campos de otra tabla). Se utiliza para introducir restricciones de integridad referencial.
<i>Tabla externa</i>	Es el nombre de la tabla que contiene el campo o los campos referenciados en refN.
<i>CampoexternoN</i>	Es el nombre del campo o de los campos de la tabla externa especificados por ref1, ref2, ..., refN.
<i>Accion</i>	Puede ser: Cascade, set null, set Default y restricted o no action



Ejemplos

```
CREATE TABLE articulos (  
  Nart INTEGER NOT NULL PRIMARY KEY,  
  Descr varchar(100), Precio FLOAT, Cant INTEGER,  
  Stock_min INTEGER, Stock_max INTEGER);
```

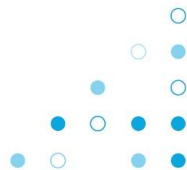
La ejecución de este comando produce como resultado la creación de la tabla artículos con seis campos; Nart de tipo entero y clave primaria de la tabla; Descr: de tipo cadena de caracteres de 100; precio de tipo FLOAT y por último los campos cant, stock_min y stock_max de tipo INTEGER.



Ejemplos

```
CREATE TABLE PROVEE  
(Nprov INTEGER NOT NULL,  
  Nart INTEGER NOT NULL,  
  Precio_venta FLOAT,  
  Constraint pkprovee primary key (Nart,Nprov),  
  Constraint fkproveedores foreign key (Nprov) references proveedores,  
  Constraint fkarticulos foreign key (Nart)  
    references articulos on delete cascade)
```

Este comando crea la tabla PROVEE con los campos Nart, Nprov y precio_venta, además tenemos como clave primaria a Nart y Nprov. La tabla PROVEE posee además dos claves Foráneas (integridad referencial), una, el atributo Nprov que hace referencia a la tabla PROVEEDORES y la otra, el atributo Nart que hace referencia a la tabla ARTICULOS y el borrado será en cascada.



Ejemplos

1) **ALTER TABLE Proveedores ADD Telefono VARCHAR (20) NOT NULL;**

La ejecución de este comando provoca que la tabla Proveedores, ya existente, incorpore una nueva columna llamada “Telefono” de tipo cadena de caracteres de 20 de longitud máxima.

2) **ALTER TABLE Proveedores DROP Direccion;**

Elimina la columna Dirección de la tabla Proveedores

3) **ALTER TABLE Proveedores ADD CONSTRAINT telefonounico UNIQUE Telefono;**

Agrega una restricción de Unique para el campo telefono de la tabla Proveedores

4) **ALTER TABLE Proveedores DROP CONSTRAINT telefonounico;**

Elimina la restricción con nombre telefonounico sobre el campo telefono de la tabla Proveedores



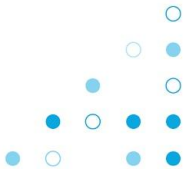
Borrado de Estructuras

El comando DROP elimina una tabla existente de una base de datos. Su sintaxis es:

DROP TABLE tabla

Donde:

Argumento	Descripción
Tabla	Nombre de la tabla que se va a eliminar



Ejemplos

1) **DROP TABLE provee;**

Elimina de la base de datos la tabla provee.

2) **DROP TABLE proveedores;**

Elimina de la base de datos la tabla proveedores.

